

Technical Documentation: RAG Customer Support Assistant

Field	Value
Document	Technical Documentation
Audience	Software Engineers / ML Engineers
Stack	Python, LangChain, LangGraph, ChromaDB, HuggingFace, OpenAI
Status	Final

1. Introduction

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture pattern that enhances Large Language Model (LLM) responses by dynamically supplying relevant, factual context retrieved from an external knowledge base. Unlike pure LLM inference — which relies solely on parametric memory learned during training — RAG grounds responses in up-to-date, domain-specific documents, dramatically reducing hallucination.

RAG = Retriever (finds relevant docs) + Generator (LLM that reads them and answers)

1.2 Why is RAG Needed?

- LLMs hallucinate facts not in their training data.
- Training data has a knowledge cutoff — new product/policy info is unknown.
- Fine-tuning is expensive; RAG is cheaper and more maintainable.
- RAG provides traceable sources (chunk + page reference) for auditing.
- Domain-specific knowledge (internal manuals, policies) is never in public LLM training.

1.3 Use Case: Customer Support Bot

A customer support bot built on RAG can ingest a company's product manuals, FAQs, and policy documents as its knowledge base. When a customer asks a question, the system retrieves the most relevant document sections and feeds them to the LLM alongside the question. The LLM then generates a precise, grounded answer — not a guess. When confidence is low or the query is sensitive (refunds, legal, fraud), the system automatically escalates to a human agent via the HITL module.

2. System Architecture Explanation

2.1 Five-Layer Architecture

Layer 1 — User Interface

CLI (main.py) exposes demo, ingest, chat, query commands. Accepts free-text user input and prints the final answer.

Layer 2 — Document Ingestion

PDF → PyPDFLoader → RecursiveCharacterTextSplitter → HuggingFace Embedder → ChromaDB.
This is a one-time pipeline that populates the vector store.

Layer 3 — Retrieval

At query time, the user's question is embedded and compared against all stored chunk vectors. Top-4 chunks by cosine similarity are returned with scores.

Layer 4 — Orchestration (LangGraph)

A 6-node StateGraph manages query lifecycle: validate → retrieve → generate → route → output. State flows as a typed Python dict.

Layer 5 — Output

Either the LLM's answer (direct route) or a HITL escalation message with ticket ID is returned to the user interface.

2.2 Component Interaction Summary

Component A	Interaction	Component B
main.py	calls run_query()	workflow.py
workflow.py	calls retrieve()	retrieval.py
retrieval.py	similarity_search()	ChromaDB
workflow.py	calls generate_answer()	llm.py
llm.py	invokes model	OpenAI / MockLLM
workflow.py	calls should_escalate()	hitl.py
hitl.py	creates ticket	_ticket_store (in-memory)

3. Design Decisions

Chunk Size: 500 chars

500 characters (~80-120 words) balances context richness vs retrieval precision. Too small, individual chunks lack meaning. Too large, retrieved chunks contain irrelevant text that dilutes the LLM prompt. 500 + 100 overlap is the sweet spot for paragraph-level retrieval.

Embedding Strategy: all-MiniLM-L6-v2

Chosen for: (a) No API cost — runs fully offline on CPU. (b) 384-dim vectors fit in memory easily. (c) MTEB benchmark shows strong semantic similarity scores. (d) normalize_embeddings=True enables cosine similarity without normalization overhead.

Top-K = 4 Chunks

Four chunks provide ~2000 chars of context — enough to answer most support questions without hitting LLM token limits. Empirically, 4-5 chunks covers 95% of answerable queries in a typical support knowledge base.

Prompt Design: Context-Only Answering

The system prompt explicitly instructs the LLM to answer ONLY from the provided context. This prevents hallucination and ensures answers are traceable to source documents. The fallback phrase 'I don't have enough information' triggers escalation.

LangGraph over Simple Chain

LangGraph's StateGraph allows conditional branching (route_node → answer vs escalate) that is

impossible in a simple sequential LangChain chain. It also makes the workflow inspectable, testable per-node, and extensible with additional nodes (memory, feedback).

ChromaDB over Pinecone/FAISS

ChromaDB requires zero external API or cloud setup, runs locally, persists to disk automatically, and integrates natively with LangChain. Ideal for internship/prototype scale. Can be swapped for Pinecone with a 2-line change for production.

4. Workflow Explanation (LangGraph)

4.1 LangGraph Overview

LangGraph extends LangChain with a graph-based execution model. A StateGraph defines nodes (Python functions) and edges (directed connections). State is a TypedDict that flows between nodes — each node receives the full state and returns an updated version. Conditional edges allow branching based on state values.

4.2 Node Responsibilities

- **input_node:** Validates and cleans the user query. Guards against empty input.
- **retrieve_node:** Embeds the query, queries ChromaDB for top-4 similar chunks, assembles the context string, and sets the `low_confidence` flag.
- **generate_node:** Constructs a two-message prompt (SystemMessage + HumanMessage) and calls the LLM. Stores the raw answer in `llm_answer`.
- **route_node:** Calls `should_escalate()` with query, confidence flag, and LLM answer. Sets `escalate` bool and `escalation_reason` in state.
- **answer_node:** Simply copies `llm_answer` to `final_answer`. Straightforward pass-through.
- **escalate_node:** Creates an `EscalationTicket`, invokes `simulate_human_response()` (demo), formats a structured escalation message, stores `ticket_id` in state.

4.3 State Transitions

Initial state → `input_node` → (user_query cleaned) → `retrieve_node` → (chunks, context, `low_confidence` set) → `generate_node` → (`llm_answer` set) → `route_node` → (escalate, reason set) → [`answer_node` | `escalate_node`] → (final_answer + optional `ticket_id` set) → END

5. Conditional Logic

5.1 Intent Detection

Intent detection in this system is rule-based and LLM-signal-based, rather than a separate ML classifier. This keeps the system lightweight while covering the most common escalation needs:

- **Keyword Intent:** Detects high-stakes topics (refund, fraud, legal) via substring matching in lowercase query. Fast $O(k)$ lookup where k = number of keywords.
- **Confidence Intent:** Low similarity scores from ChromaDB indicate the knowledge base does not cover the topic. This is a proxy for 'out-of-domain' intent.
- **Self-Aware Intent:** The LLM's own uncertainty ('I don't have enough information') is treated as an escalation signal.
- **Complexity Intent:** Queries > 30 words likely involve multi-step reasoning that the RAG system may not handle well.

5.2 Routing Decision Summary

Trigger	Signal	Route
Sensitive keyword	keyword in query.lower()	ESCALATE
Low retrieval confidence	best_score < 0.6	ESCALATE
LLM uncertainty	'don't have enough information'	ESCALATE
Complex query	word_count > 30	ESCALATE
None of the above	—	ANSWER

6. HITL Implementation

6.1 Role of Human Intervention

Human-in-the-Loop (HITL) is the safety net for cases where the automated RAG pipeline cannot provide a reliable answer. It ensures that: (a) customers with urgent or sensitive issues receive qualified human attention; (b) the system fails gracefully when knowledge base coverage is insufficient; (c) complex legal/financial queries are never auto-answered with potentially incorrect information.

6.2 HITL Benefits

- Prevents incorrect answers on high-stakes queries (refunds, legal, safety)
- Provides fallback when the vector store has coverage gaps
- Creates a natural data collection loop — human responses can train future models
- Improves customer trust: complex cases get human empathy and judgment
- Ticket system enables SLA tracking and support quality monitoring

6.3 HITL Limitations

- Introduces latency — human response may take minutes to hours
- Requires human agent availability and training costs
- In demo mode, simulate_human_response() returns canned text — not real intelligence
- Escalation criteria are heuristic — may over-escalate or under-escalate edge cases

7. Challenges & Trade-offs

Retrieval Accuracy vs Speed

More chunks retrieved (higher k) improves answer completeness but increases LLM prompt size and cost. We use k=4 as a pragmatic balance. A hybrid BM25 + dense retrieval (e.g., Cohere Rerank) would improve precision at the cost of complexity.

Chunk Size vs Context Quality

Large chunks (1000+ chars) provide rich context but dilute the specific answer. Small chunks (200 chars) are precise but may lack surrounding context. 500 chars + 100 overlap is the empirically validated default in the LangChain ecosystem.

Cost vs Performance

GPT-4 produces significantly better answers than GPT-3.5-turbo but costs 20x more. For production, use GPT-3.5-turbo with carefully engineered prompts for ~90% of queries, escalating the remaining 10% (complex/uncertain) to GPT-4 or humans.

Local Embeddings vs API Embeddings

HuggingFace (local) eliminates per-query embedding cost but is slower on CPU (~50ms vs ~10ms for OpenAI API). For high-throughput production, switch to OpenAI Embeddings or add a GPU.

HITL Latency vs Automation Rate

Tighter escalation thresholds → more human tickets → better safety but higher cost and slower response. Looser thresholds → more automation → higher risk of wrong answers. Tune ESCALATION_KEYWORDS and RELEVANCE_THRESHOLD based on real query data.

8. Testing Strategy

8.1 Testing Approach

Testing is structured across three levels: unit testing of individual modules, integration testing of the end-to-end pipeline, and behavioral testing with representative query sets.

Level	Scope	Tools	Example
Unit	Individual functions	pytest	test chunk_documents() returns correct count
Integration	Module-to-module interactions	pytest + fixtures	test full ingest → retrieve pipeline
Behavioral	End-to-end query scenarios	run_query() + assertion	test escalation triggers on 'refund' keyword
Offline Demo	No PDF, synthetic KB	main.py demo	5 scripted queries covering all paths

8.2 Sample Test Queries

Query	Expected Route	Expected Behaviour
What is your return policy?	ANSWER	LLM answers from KB chunk about returns
How do I reset my password?	ANSWER	LLM answers from KB chunk about password reset
What are your support hours?	ANSWER	LLM answers from KB chunk about business hours
I need a refund — this is fraud!	ESCALATE	Keyword 'refund'+'fraud' triggers HITL
What is quantum entanglement?	ESCALATE	Low confidence — not in KB
Can you explain in detail why...	ESCALATE	Word count > 30 threshold

9. Future Enhancements

Multi-Document Support

Add a document_id field to chunk metadata. Route queries to the relevant document collection based on detected topic. Enable knowledge base versioning so new PDFs can be added without resetting the vector store.

Feedback Loop

After each answer, collect thumbs-up/thumbs-down from users. Store feedback alongside the query-context-answer triplet. Use feedback data to: (a) retrain the embedding model, (b) tune escalation thresholds, (c) identify knowledge gaps.

Conversation Memory

Add a memory_node to the LangGraph that stores past turns. Use LangChain's ConversationBufferWindowMemory to maintain context across multi-turn dialogues. Integrate conversation history into the LLM prompt.

Web Deployment

Wrap run_query() in a FastAPI endpoint. Build a React/Next.js chat UI. Deploy on AWS/GCP with Docker. Use Redis for session state and query caching. ChromaDB → Pinecone for distributed vector search.

Hybrid Retrieval

Combine dense (embedding) retrieval with sparse (BM25 keyword) retrieval. Apply a cross-encoder reranker (Cohere Rerank) to re-score top-10 candidates. Improves precision for queries with specific named entities or codes.

Observability

Add LangSmith tracing to log every graph execution. Monitor: retrieval scores, escalation rate,

LLM latency, token usage, cost per query. Alert when escalation rate spikes (knowledge base coverage degradation).